

Sliding Window Log Algorithm (Rate Limiting)

The **Sliding Window Log Algorithm** solves the biggest weakness of the Fixed Window Counter algorithm:

None

Window boundary burst problem

Instead of counting requests in rigid blocks, it checks requests in a **rolling time window**.

This makes rate limiting much more accurate.

1. Core Idea

Store timestamps of requests and allow only a certain number within the latest time window.

Example:

None

Limit = 2 requests / minute

At any moment, only last 60 seconds matter

2. How It Works

Step 1: Store Request Timestamps

Each accepted/rejected request timestamp is logged.

Usually stored in:

- Redis Sorted Sets
- In-memory ordered structures

None

```
[1:00:01, 1:00:30, 1:00:50]
```

Step 2: Remove Outdated Entries

When new request arrives:

None

```
Delete timestamps older than current_time - window_size
```

Only recent timestamps remain.

Step 3: Insert New Timestamp

Add incoming request timestamp to log.

Step 4: Check Count

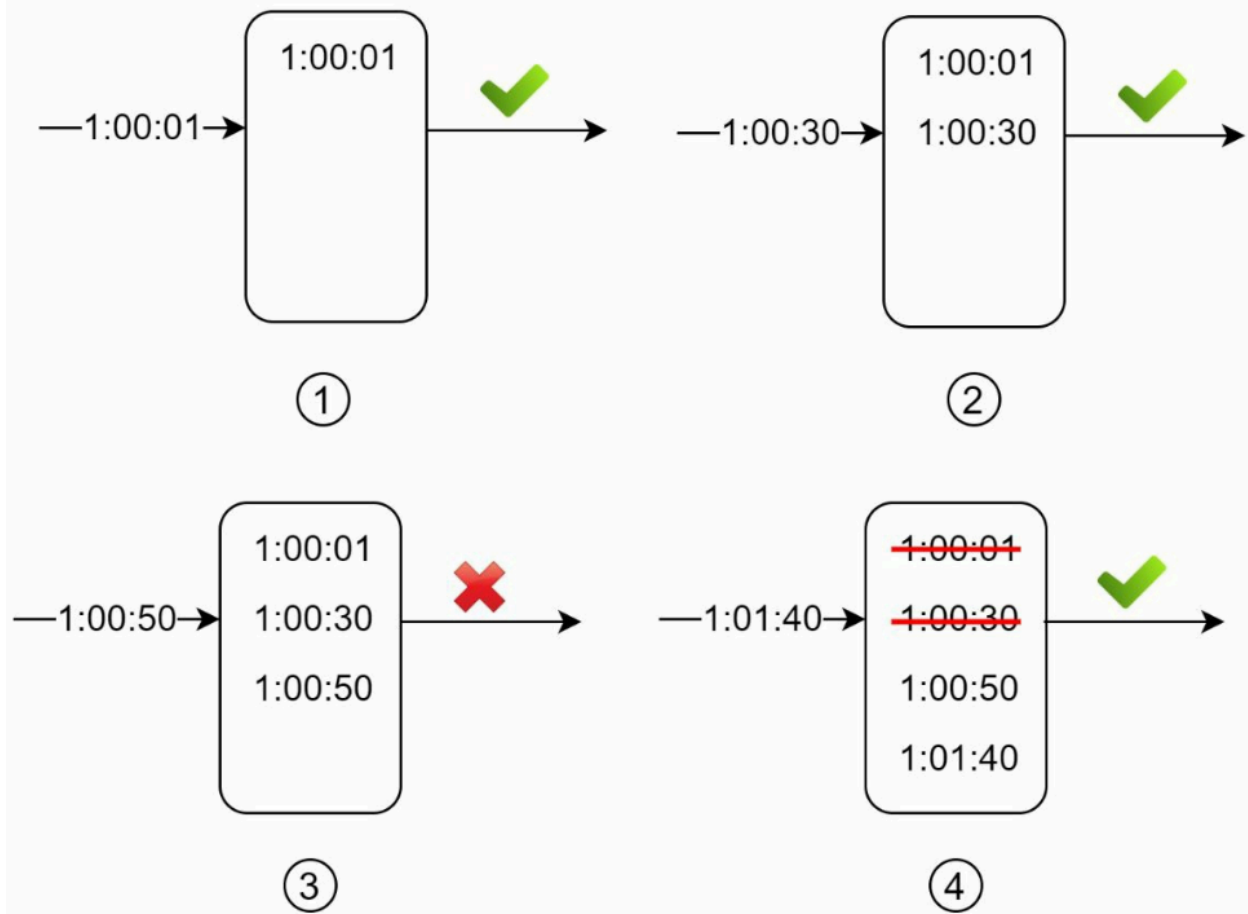
None

If $\text{log size} \leq \text{limit} \rightarrow \text{allow request}$

Else $\rightarrow \text{reject request}$

3. Example

Allow 2 requests per minute



System limit:

None

2 requests / minute

Request 1 at 1:00:01

Log was empty:

None

[1:00:01]

Allowed

Request 2 at 1:00:30

Insert timestamp:

None

[1:00:01, 1:00:30]

Size = 2

Allowed

Request 3 at 1:00:50

Insert timestamp:

None

[1:00:01, 1:00:30, 1:00:50]

Size = 3 > limit

Rejected

(But timestamp may remain stored depending on implementation)

Request 4 at 1:01:40

Current rolling window:

None

[1:00:40, 1:01:40)

Outdated timestamps removed:

None

Remove 1:00:01

Remove 1:00:30

Remaining:

None

[1:00:50, 1:01:40]

Size = 2

Allowed

4. Why It Is Better Than Fixed Window

Fixed window can allow bursts at boundary.

Sliding window checks real rolling interval:

None

At any 60-second period, limit is enforced accurately

5. Advantages

- Very accurate rate limiting
- Prevents edge-window spikes
- Fair request distribution
- Strong control for abuse prevention

6. Disadvantages

Higher Memory Usage

Need to store many timestamps.

None

High traffic user = many stored timestamps

More Expensive Operations

Need:

- Delete old entries
- Insert new entry
- Count current entries

Harder to Scale

Distributed timestamp consistency may be needed.

7. Best Use Cases

- Login brute-force protection
- Payment APIs
- Premium APIs needing precise fairness
- Security-sensitive systems
- User-facing APIs with strict quotas

8. Implementation Notes

Common implementation:

None

Redis Sorted Set:

`score = timestamp`

`member = request id`

Operations:

- ZREMRANGEBYSCORE (remove old)
- ZADD (insert)
- ZCARD (count)

9. Comparison with Fixed Window

Feature	Fixed Window	Sliding Window Log
Simplicity	Very High	Medium
Accuracy	Low	High
Boundary Burst	Yes	No
Memory Usage	Low	High

Mental Model

None

Sliding Window Log = Keep recent request history, enforce limit on moving timeline

One-Line Summary

Sliding Window Log Algorithm rate-limits requests by storing timestamps and counting only those within the latest rolling time window, providing highly accurate and fair throttling at the cost of higher memory usage.